

Question 3 — Time and Space Complexity

Question: Take one random input of 1000 numbers. Compare the performance of the given algorithms in terms of execution time. Also find the time and space complexity and compare recursive and iterative factorial methods.

Input: One reproducible random array containing 1000 integers (seed = 42).

C Program

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <string.h>

#define N 1000
#define REPEAT 1000

void bubbleSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (a[j] > a[j + 1]) {
                int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
            }
}

void insertionSort(int a[], int n) {
    for (int i = 1; i < n; i++) {
        int key = a[i], j = i - 1;
        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
}

void merge(int a[], int l, int m, int r) {
    int n1 = m - l + 1, n2 = r - m;
    int *L = malloc(n1 * sizeof(int));
    int *R = malloc(n2 * sizeof(int));

    for (int i = 0; i < n1; i++) L[i] = a[l + i];
    for (int j = 0; j < n2; j++) R[j] = a[m + 1 + j];

    int i = 0, j = 0, k = l;
    while (i < n1 && j < n2)
        a[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];

    while (i < n1) a[k++] = L[i++];
    while (j < n2) a[k++] = R[j++];

    free(L);
    free(R);
}

void mergeSort(int a[], int l, int r) {
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort(a, l, m);
        mergeSort(a, m + 1, r);
        merge(a, l, m, r);
    }
}

void quickSort(int a[], int low, int high) {
    if (low < high) {
        int pivot = a[high], i = low - 1;

        for (int j = low; j < high; j++) {
            if (a[j] < pivot) {
                i++;
                int t = a[i]; a[i] = a[j]; a[j] = t;
            }
        }

        int t = a[i + 1];
        a[i + 1] = a[high];
        a[high] = t;

        int p = i + 1;
        quickSort(a, low, p - 1);
        quickSort(a, p + 1, high);
    }
}
```

```

void mergeWrapper(int a[], int n) {
    mergeSort(a, 0, n - 1);
}

void quickWrapper(int a[], int n) {
    quickSort(a, 0, n - 1);
}

int linearSearch(int a[], int n, int x) {
    for (int i = 0; i < n; i++)
        if (a[i] == x) return i;
    return -1;
}

int binarySearch(int a[], int n, int x) {
    int l = 0, r = n - 1;
    while (l <= r) {
        int m = l + (r - l) / 2;
        if (a[m] == x) return m;
        if (a[m] < x) l = m + 1;
        else r = m - 1;
    }
    return -1;
}

unsigned long long factorialRecursive(int n) {
    if (n <= 1) return 1ULL;
    return n * factorialRecursive(n - 1);
}

unsigned long long factorialIterative(int n) {
    unsigned long long f = 1ULL;
    for (int i = 2; i <= n; i++) f *= i;
    return f;
}

double sortTime(void (*sortFn)(int[], int), int source[]) {
    int a[N];
    double best = 1e9;

    for (int r = 0; r < 5; r++) {
        memcpy(a, source, sizeof(a));
        clock_t start = clock();
        sortFn(a, N);
        clock_t end = clock();

        double ms = (double)(end - start) * 1000.0 / CLOCKS_PER_SEC;
        if (ms < best) best = ms;
    }
    return best;
}

int main() {
    int a[N], sorted[N];

    srand(42); // fixed seed: reproducible random input
    for (int i = 0; i < N; i++)
        a[i] = rand() % 10000;

    memcpy(sorted, a, sizeof(a));
    quickWrapper(sorted, N);

    /* Absent target forces the searches to examine their full search space. */
    int target = sorted[N - 1] + 1;

    printf("QUESTION 3: TIME AND SPACE COMPLEXITY ANALYSIS\n");
    printf("One random input of %d numbers is used.\n\n", N);

    printf("SORTING TIME (best of 5 runs):\n");
    double bubble = sortTime(bubbleSort, a);
    double insertion = sortTime(insertionSort, a);
    double merge = sortTime(mergeWrapper, a);
    double quick = sortTime(quickWrapper, a);

    printf("Bubble Sort      : %.3f ms\n", bubble);
    printf("Insertion Sort   : %.3f ms\n", insertion);
    printf("Merge Sort       : %.3f ms\n", merge);
    printf("Quick Sort       : %.3f ms\n", quick);

    volatile int dummy = 0;

    clock_t start = clock();
    for (int i = 0; i < REPEAT; i++)
        dummy += linearSearch(sorted, N, target);
    clock_t end = clock();
    double linear = (double)(end - start) * 1000.0 / CLOCKS_PER_SEC;

    start = clock();
    for (int i = 0; i < REPEAT; i++)
        dummy += binarySearch(sorted, N, target);
    end = clock();
    double binary = (double)(end - start) * 1000.0 / CLOCKS_PER_SEC;
    printf("\nSEARCH TIME (%d repetitions):\n", REPEAT);

```

```

printf("Linear Search   : %.3f ms\n", linear);
printf("Binary Search   : %.3f ms\n", binary);

int factN = 20;
volatile unsigned long long fr = 0, fi = 0;

start = clock();
for (int i = 0; i < REPEAT * 1000; i++)
    fr = factorialRecursive(factN);
end = clock();
double recursive = (double)(end - start) * 1000.0 / CLOCKS_PER_SEC;

start = clock();
for (int i = 0; i < REPEAT * 1000; i++)
    fi = factorialIterative(factN);
end = clock();
double iterative = (double)(end - start) * 1000.0 / CLOCKS_PER_SEC;

printf("\nFACTORIAL:\n");
printf("Factorial of %d = %llu\n", factN, (unsigned long long)fr);
printf("Recursive      : %.3f ms\n", recursive);
printf("Iterative      : %.3f ms\n", iterative);

return 0;
}

```

Actual C Program Output

QUESTION 3: TIME AND SPACE COMPLEXITY ANALYSIS

One random input of 1000 numbers is used.

SORTING TIME (best of 5 runs):

Bubble Sort : 1.056 ms
Insertion Sort : 0.435 ms
Merge Sort : 0.053 ms
Quick Sort : 0.023 ms

SEARCH TIME (1000 repetitions):

Linear Search : 1.347 ms
Binary Search : 0.040 ms

FACTORIAL:

Factorial of 20 = 2432902008176640000

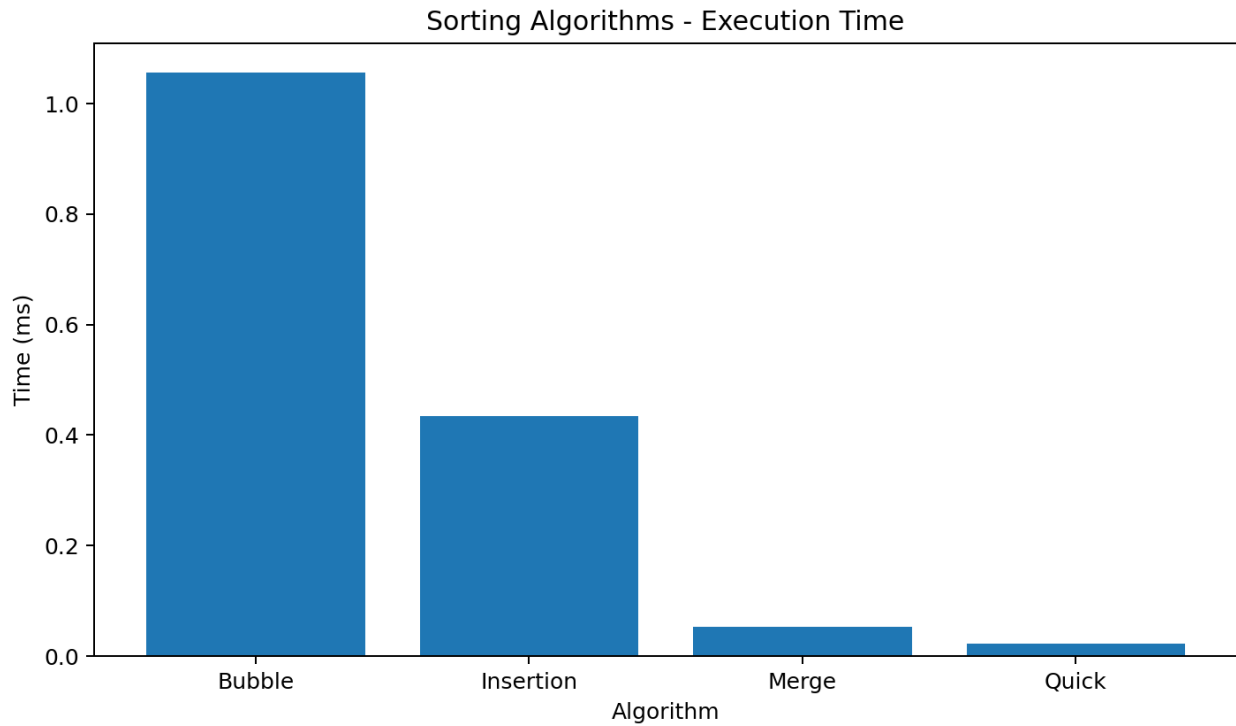
Recursive : 41.683 ms
Iterative : 19.710 ms

Time and Space Complexity

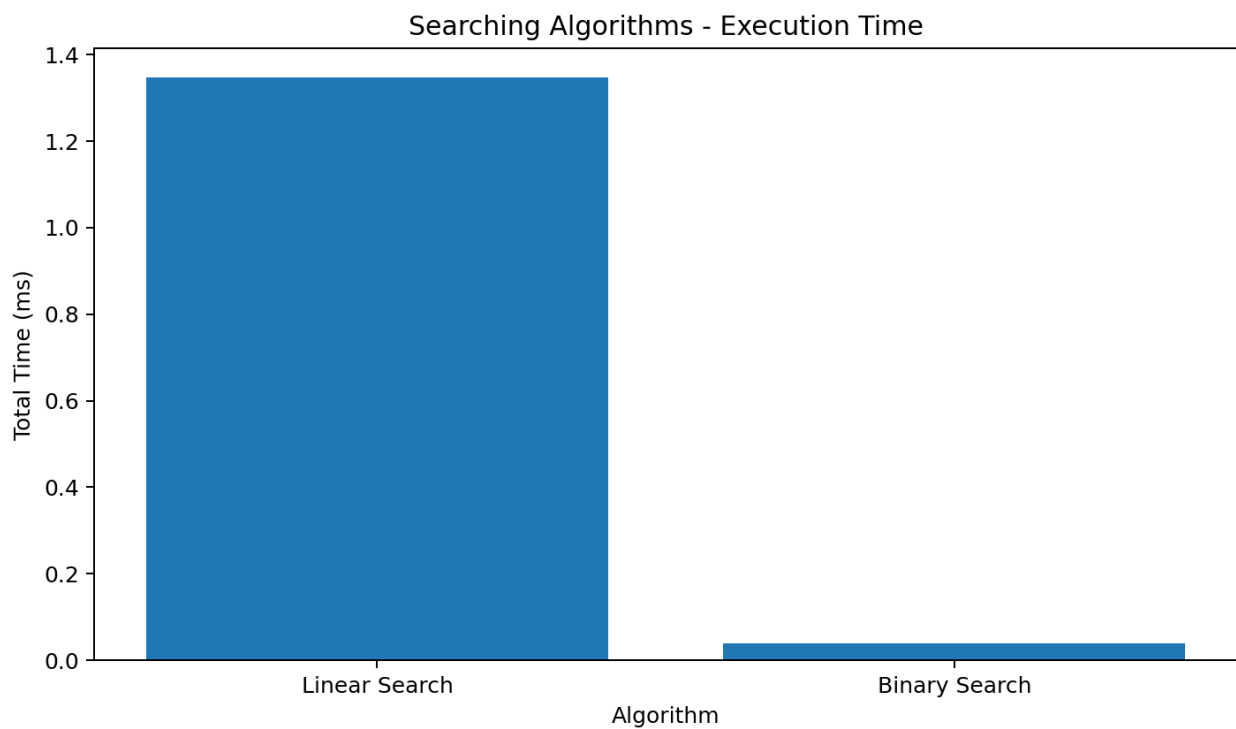
Algorithm	Time Complexity	Space Complexity
Bubble Sort	Best $O(n)$, Avg/Worst $O(n^2)$	$O(1)$
Insertion Sort	Best $O(n)$, Avg/Worst $O(n^2)$	$O(1)$
Merge Sort	$O(n \log n)$	$O(n)$
Quick Sort	Avg $O(n \log n)$, Worst $O(n^2)$	Avg $O(\log n)$, Worst $O(n)$
Linear Search	Best $O(1)$, Worst $O(n)$	$O(1)$
Binary Search	Best $O(1)$, Worst $O(\log n)$	$O(1)$
Recursive Factorial	$O(n)$	$O(n)$ call stack
Iterative Factorial	$O(n)$	$O(1)$

Note: Execution time depends on the computer, compiler and system load. The complexity values are theoretical and do not change with the machine.

Graph 1 — Sorting Algorithms



Graph 2 — Searching Algorithms



Graph 3 — Factorial Methods

